

Reviewer Pack — Minimal Geometric Entropy Correlation with Communication Ran...

Entry e36dca15c87b · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-01 03:15:29 UTC

Minimal Geometric Entropy Correlation with Communication Rank Growth in Planar Graphs

Entry ID: e36dca15c87b Verdict: INCONCLUSIVE Recorded: 2026-06-01 03:15:29 UTC

1. Conjecture

Field A (mathematical branch): Geometric Entropy Theory **Field B** (complexity object): Communication Complexity

Statement:

For any given planar graph G , the geometric entropy $H(G)$ of its communication complexity tree is linearly correlated with its communication rank growth rate $r(G)$, such that $H(G) = \Theta(r(G))$.

Rationale (proposer’s reasoning):

Geometric Entropy Theory provides a measure of the information content in the structure of graphs. Correlating this with the communication rank could potentially reveal new insights into the inherent complexity of communication tasks.

Taxonomy category: GeometricEntropy_x_CommunicationComplexity (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 24f1e2e3a44dc2ae

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if at least 80% of the computed pairs $(H(G), r(G))$ from random planar graphs G show a linear correlation with a slope within ± 3 standard deviations of the expected slope, and no seed graph produces $H(G)$ or $r(G)$ outside the range $[0.1, 10]$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	1.00	UNCERTAIN	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "geometric entropy" AND "communication rank growth" AND planar graphs"
- "communication complexity tree" AND "geometric entropy" AND linear correlation" - " $\Theta(r(G))$ "
AND communication complexity AND geometric entropy

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_planar_graph(n):
        if n < 3:
            return None

        # Generate a random planar graph using the method of adding edges to a base cycle
        nodes = list(range(1, n + 1))
        edges = [(nodes[i], nodes[(i + 1) % n]) for i in range(n)]

        for _ in range(n):
            u = random.choice(nodes)
            v = random.choice(nodes)
            if (u, v) not in edges and (v, u) not in edges:
                edges.append((u, v))
                # Check planarity condition
                if not is_planar(edges):
                    return None

        return edges

    def is_planar(edges):
        if len(edges) > 3 * len(nodes) - 6:
            return False

    def dfs(v, parent, visited, rec_stack):
        visited[v] = True
        rec_stack[v] = True
```

```

    for u in range(1, n + 1):
        if (v, u) in edges or (u, v) in edges:
            if not visited[u]:
                if dfs(u, v, visited, rec_stack):
                    return True
            elif u != parent and rec_stack[u]:
                return True

    rec_stack[v] = False
    return False

visited = [False] * (n + 1)
rec_stack = [False] * (n + 1)

for v in range(1, n + 1):
    if not visited[v]:
        if dfs(v, -1, visited, rec_stack):
            return False

return True

def geometric_entropy(edges):
    # Simplified entropy calculation based on the number of edges
    return len(edges) / (n * (n - 1))

def communication_rank_growth_rate(edges):
    # Simplified rank growth rate calculation based on the number of edges
    return len(edges)

n_values = [5, 10, 15, 20, 30, 40]
results = []

for n in n_values:
    graph = generate_planar_graph(n)
    if graph is None:
        continue

    H_G = geometric_entropy(graph)
    r_G = communication_rank_growth_rate(graph)

    if not (0.1 <= H_G <= 10) or not (0.1 <= r_G <= 10):
        return {
            "metric_name": "geometric_entropy",
            "metric_value": H_G,
            "instances_tested": len(results),
            "n_max": n,
            "conjecture_holds": False,
            "counterexample": "H(G) or r(G) outside range [0.1, 10]"
        }

    results.append((H_G, r_G))

if not results:
    return {
        "metric_name": "geometric_entropy",
        "metric_value": None,

```

```

        "instances_tested": len(results),
        "n_max": max(n_values) if n_values else 1,
        "conjecture_holds": False,
        "counterexample": "No valid graphs generated"
    }

H_Gs, r_Gs = zip(*results)
slope, intercept = linear_regression(H_Gs, r_Gs)

return {
    "metric_name": "geometric_entropy",
    "metric_value": slope,
    "instances_tested": len(results),
    "n_max": max(n_values) if n_values else 1,
    "conjecture_holds": abs(slope - 1) <= 3 * stdev(r_Gs),
    "counterexample": ""
}

def linear_regression(x, y):
    n = len(x)
    sum_x = sum(x)
    sum_y = sum(y)
    sum_xy = sum(xi * yi for xi, yi in zip(x, y))
    sum_xx = sum(xi ** 2 for xi in x)

    slope = (n * sum_xy - sum_x * sum_y) / (n * sum_xx - sum_x ** 2)
    intercept = (sum_y - slope * sum_x) / n

    return slope, intercept

def stdev(values):
    mean = sum(values) / len(values)
    variance = sum((x - mean) ** 2 for x in values) / len(values)
    return math.sqrt(variance)

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_slope = sum(r["metric_value"] for r in results if r["conjecture_holds"]) / sum(1 for r in results if r["conjecture_holds"])
    std_slope = stdev([r["metric_value"] for r in results if r["conjecture_holds"]])
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_slope} std={std_slope} support_fraction={support_fraction}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_slope} std={std_slope} support_fraction={support_fraction}")
    else:
        first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)

```

```

counterexample = next((r["counterexample"] for r in results if r["counterexample"]), "")
print(f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={first_fa

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_14d76469.py", line 146, in <module>
    result = run_trial(seed)
              ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_14d76469.py", line 81, in run_trial
    graph = generate_planar_graph(n)
            ^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_14d76469.py", line 35, in generate_planar_graph
    if not is_planar(edges):
           ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_14d76469.py", line 41, in is_planar
    if len(edges) > 3 * len(nodes) - 6:
                          ^^^^^
NameError: name 'nodes' is not defined

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which prevents us from evaluating the conjecture’s support or falsification. | next: Investigate and fix the crash in the test code to proceed with the evaluation of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	26518
2	preregistration	ollama_remote	glm4:latest	0	0	9734
3	novelty	ollama_remote	glm4:latest	0	0	9664
4	novelty	ollama_remote	glm4:latest	0	0	10332
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	20748
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	19711
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14526
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15617

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
9	judge	ollama_remote	glm4:latest	0	0	15697

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 142546 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in research/audit/e36dca15c87b.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/e36dca15c87b.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/e36dca15c87b.tar.gz (if generated)